

KARADENIZ TECHNICAL UNIVERSITY — GRADUATE SCHOOL OF NATURAL AND APPLIED SCIENCES

Constraint Programming

Soft Constraints and Optimization

Week 8

Soft Constraints and Optimization

Assoc. Prof. Tolga Berber

KTU Faculty of Science, Department of Computer Science · https://avesis.ktu.edu.tr/tberber

Learning phases: 1 • Pre-instruction 2 • Content 3 • Participation 4 • Assessment 5 • Follow-through

1 • Pre-instruction Why It Matters?

Week 1 asked you to keep two shared-author talks, T1 and T2, out of the same slot – solvable, room to spare.

- Now two more talks join the program, T4 and T5, and between them some shared-author pair is unavoidably double-booked, no matter how you assign the five talks to two slots
- Every hard-constraint CSP solver you built through Week 6 just reports `INFEASIBLE` and stops – useless to an organizer who still needs a program printed

Do you tell them “no valid schedule exists,” or hand them the schedule that breaks the fewest rules? This week builds the machinery for the second answer.

This Week

◎ Learning Objectives

By the end of this week, you will be able to:

1. Define an over-constrained problem and distinguish Max-CSP, weighted CSP (WCSP) and valued CSP
2. Formulate a CP-SAT objective from reified violation Booleans, linked with `OnlyEnforceIf`
3. Carry out lexicographic optimization and determine Pareto dominance to build a Pareto front
4. Build a complete soft-constraint model combining hard coverage rules with weighted soft preferences
5. Translate a real over-constrained scenario into a verified, running CP-SAT soft-constraint program

SECTION 1

Unit 1 — Over-Constrained Problems: Max-CSP, WCSP, Valued CSP

2 • Content From INFEASIBLE to “Least Bad”

- The five-talk instance has four shared-author constraints: T1–T3, T1–T4, T3–T4, T2–T5
- T1, T3 and T4 form a triangle of pairwise “must differ” rules – with only two slots, some pair inside it is forced together, no matter the assignment

Definition: Over-constrained problem

A constraint satisfaction problem whose hard constraints cannot all be satisfied by any complete assignment.

Max-CSP: Count Violations, Minimize Them

Definition: Max-CSP

The relaxation of an over-constrained CSP in which every constraint becomes soft and the goal is the assignment satisfying the largest number of constraints, equivalently violating the fewest.

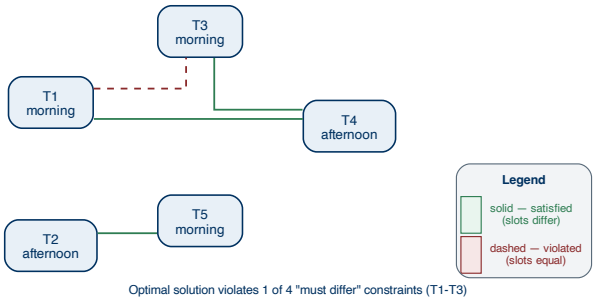
Analogy

An over-booked event venue trying to seat every guest's request faces the same situation: when two guests both insist on the same table and only one can have it, the venue does not cancel the event – it accepts the seating plan that honours the most requests, or costs the fewest refused requests.

Worked by Hand: One Violation vs. Two

- **Assignment A:** T1=morning, T2=morning, T3=afternoon, T4=afternoon, T5=afternoon – only T3–T4 clash, 1 violation
- **Assignment B:** T1=morning, T3=morning, T4=morning, T2=afternoon, T5=afternoon – T1–T3 and T1–T4 both clash, 2 violations
- Neither is a “clean” CSP solution – but Max-CSP strictly prefers A, since 1 violation beats 2

Real Optimum: The Five-Talk Constraint Graph



The real CP-SAT-optimal assignment: T1 and T3 share the morning slot – the only violated “must differ” edge; the other three edges are satisfied.

Weighted CSP: Not Every Violation Costs the Same

Definition: Weighted CSP (WCSP)

A generalization of Max-CSP in which each constraint carries a cost for being violated; the goal is the assignment minimizing total violation cost.

Analogy

A household budget with priority-weighted bills makes the same trade-off: rent is unavoidable, never cut; a streaming subscription is discretionary, and cutting it "costs" less regret than cutting another. The family minimizes total regret, not a raw count of cuts.

Max-CSP is exactly the WCSP special case where every weight equals 1.

Valued CSP: the Umbrella Term

Definition: Valued CSP

The general framework in which constraint satisfaction is a graded value rather than a yes/no, and the objective combines those values; WCSP is its most common instance in this course.

- This week works concretely with Max-CSP and WCSP throughout
- A WCSP (or Max-CSP) *is* a constraint optimization problem (Week 1) whose objective is specifically “minimize weighted constraint violation” – Week 1’s CSP-vs-COP distinction, now with its first worked instance

3 • Participation Quick Check: One Violation or Two?

List the five-talk instance's shared-author constraints (T1–T3, T1–T4, T3–T4, T2–T5). Find one assignment violating exactly one of them, and one violating two. Which does Max-CSP prefer, and why?

T1=morning, T2=morning, T3=afternoon, T4=afternoon, T5=afternoon violates only T3–T4 (1 violation). T1=morning, T3=morning, T4=morning, T2=afternoon, T5=afternoon violates T1–T3 and T1–T4 (2 violations). Max-CSP counts violated constraints and prefers fewer – 1 beats 2, even though neither is a clean CSP solution.

SECTION 2

Unit 2 — Objective Functions in CP-SAT

2 • Content From `gurobipy` to CP-SAT

Week 7's `model.setObjective(...)` said "minimize total production cost." This unit asks the same question inside CP-SAT.

- `model.Minimize(expr)` / `model.Maximize(expr)` – direct analogue of Week 7's `gurobipy` objective, `expr` must be linear over the model's variables
- `cp_model.OPTIMAL` means the optimum is *proven*; `cp_model.FEASIBLE` means a time limit cut search short – exactly Week 7's status vocabulary, reused unchanged

Reification: Giving a Condition a Variable of Its Own

Definition: Reification

Turning a constraint's truth value into a Boolean decision variable that other parts of the model, in particular the objective, can reference.

Analogy

A household checklist scorecard: for each house rule, a checkbox flips to "broken" only if that rule was actually broken this week, and the weekly score is the sum of flipped checkboxes – a reified violation variable is exactly that one checkbox, tied to one specific rule.

Code: Reifying T1–T3's Violation Boolean

```
slots = {talk: model.NewIntVar(0, 1, f"slot_{talk}") for talk in
    TALKS}
return slots
```

```
violations = {}
for talk_a, talk_b in PAIRS:
    viol = model.NewBoolVar(f"viol_{talk_a}_{talk_b}")
    model.Add(slots[talk_a] == slots[talk_b]).OnlyEnforceIf(viol)
    model.Add(slots[talk_a] != slots[talk_b]).OnlyEnforceIf(viol.Not()
        )
    violations[(talk_a, talk_b)] = viol
return violations
```

The first `model.Add(...)` forces `viol` to 1 when the two talks clash; the second forces it to 0 when they do not – *both* directions are needed.

Common Pitfall: One Direction Is Not Enough

Common Pitfall

Leaving out `OnlyEnforceIf(viol.Not())` lets the solver set `viol = 0` even when the talks genuinely clash – nothing in the model forbids it, and `model.Minimize()` will happily exploit that freedom to make the objective look smaller than it really is, silently undercounting violations.

Code: The Weighted-Violation Objective

```
weight = 1
total_violations = sum(weight * viol for viol in violations.values())
model.Minimize(total_violations)
return total_violations
```

For the Max-CSP case, every weight equals 1 – Unit 1's observation, now written directly into code.

Code: Solving and Reading Back

```
solver = cp_model.CpSolver()
solver.parameters.num_workers = 1
solver.parameters.random_seed = 42
status = solver.Solve(model)
assignment = {talk: solver.Value(slots[talk]) for talk in TALKS}
total = solver.Value(total_violations)
print(f"Status: {solver.StatusName(status)}")
for talk in TALKS:
   print(f"{talk}: {SLOT_NAMES[assignment[talk]]}")
print(f"Total violations: {total}")
```

One solve call reports both the optimal assignment and the total violation count.

Output: The Weighted Objective, For Real

Expected Output

```
Status: OPTIMAL
T1: morning
T2: afternoon
T3: morning
T4: afternoon
T5: morning
Total violations: 1
```

`OPTIMAL` means CP-SAT has *proven* no assignment achieves fewer than 1 violation – matching Figure 1's T1–T3 clash exactly.

3 • Participation Quick Check: Write Both Directions

For `viol_T1_T3`, write the two `OnlyEnforceIf` calls linking it to `slot[T1] == slot[T3]`.

`model.Add(slot[T1] == slot[T3]).OnlyEnforceIf(viol_T1_T3)` forces the Boolean to 1 whenever the talks clash; `model.Add(slot[T1] != slot[T3]).OnlyEnforceIf(viol_T1_T3.Not())` forces it to 0 whenever they do not – both directions are needed, or the objective can silently undercount violations.

SECTION 3

Unit 3 — Lexicographic and Multi-Objective Optimization, Pareto Fronts

2 • Content Why a Single Number Can Hide a Trade-off

- Minimizing total violations alone might return a schedule that is numerically optimal but crams four of five talks into the afternoon – technically correct, not what any organizer wants
- Week 1 Unit 1's own feedback foreshadowed a second objective: "minimize the number of afternoon slots used"
- Two objectives will not always agree on what "best" means

Lexicographic Optimization: Rank, Then Pin

Definition: Lexicographic optimization

Optimizing a ranked sequence of objectives in strict priority order, pinning each objective's optimal value as a hard constraint before optimizing the next.

Analogy

Ranking apartment-hunting priorities in strict order: first, among apartments within budget, find the cheapest; only among those tied on price, pick the shortest commute. The second criterion never overrides the first.

Code: Stage 1 — Minimize Total Violations

```python
model.Minimize(total_violations)
solver = cp_model.CpSolver()
solver.parameters.num_workers = 1
solver.parameters.random_seed = 42
solver.Solve(model)
v1_star = solver.Value(total_violations)
print(f"Stage 1 optimal total violations: {v1_star}")
```

Exactly Unit 2's objective, reused as-is; the optimal value is recorded as `v1_star` before stage 2 starts.

Code: Stage 2 — Pin Stage 1's Optimum

```
model.Add(total_violations == v1_star)
afternoon_count = sum(slots[talk] for talk in TALKS)
```

A *fresh* model: the same variables, rebuilt, with stage 1's optimum pinned as a hard constraint.

Code: Stage 2 — Minimize Afternoon Count

```
model.Minimize(afternoon_count)
solver = cp_model.CpSolver()
solver.parameters.num_workers = 1
solver.parameters.random_seed = 42
solver.Solve(model)
afternoon_star = solver.Value(afternoon_count)
print(f"Stage 2 optimal afternoon count (violations pinned at {
    v1_star}): {afternoon_star}")
for talk in TALKS:
   print(f"{talk}: {SLOT_NAMES[solver.Value(slots[talk])]}")
```

Only now, with the pin in place, is the second objective declared and solved.

Output: Both Stages, For Real

Expected Output

```
Stage 1 optimal total violations: 1
Stage 2 optimal afternoon count (violations pinned at 1): 2
T1: morning
T2: morning
T3: morning
T4: afternoon
T5: afternoon
```

Stage 2's afternoon count of 2 never trades away stage 1's proven optimum of 1 violation.

Pareto Dominance and the Pareto Front

Definition: Pareto dominance

A candidate solution A dominates candidate B if A is at least as good as B on every objective and strictly better on at least one.

Definition: Pareto front

The set of candidate solutions that no other candidate dominates, under a given set of objectives.

Analogy: Comparing Apartment Listings

Analogy

Compare listings on price and commute time: a listing is only worth shortlisting if no other listing beats it on both at once. A cheaper-but-farther listing and a pricier-but-closer listing can both stay on the shortlist – neither dominates the other.

Unlike lexicographic optimization, a Pareto front hands the decision-maker every reasonable trade-off point, not a single winner.

Code: Enumerate Every Feasible Candidate

```
def __init__(self, slots: dict[str, cp_model.IntVar]) -> None:
    super().__init__()
    self._slots = slots
    self.candidates: list[tuple[int, int]] = []

def OnSolutionCallback(self) -> None:
    assignment = {talk: self.Value(self._slots[talk]) for talk in TALKS}
    violations = sum(1 for a, b in PAIRS if assignment[a] == assignment[b
        ])
    afternoon = sum(assignment.values())
    self.candidates.append((violations, afternoon))
```

Week 1 Unit 4's enumerate-all-solutions technique, reused as a callback recording each candidate's (violations, afternoon) pair.

Code: The Dominance Test

```
at_least_as_good = candidate_a[0] <= candidate_b[0] and candidate_a
    [1] <= candidate_b[1]
strictly_better = candidate_a[0] < candidate_b[0] or candidate_a[1] <
     candidate_b[1]
return at_least_as_good and strictly_better
```

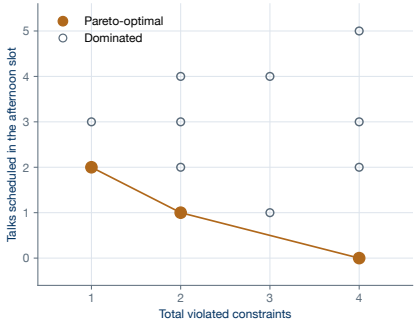
A pure Python function matching the dominance definition almost word for word – no CP-SAT call needed here.

Code: Filtering the Pareto Front

```
unique_candidates = sorted(set(candidates))
front = [
   candidate
   for candidate in unique_candidates
   if not any(
      dominates(other, candidate) for other in unique_candidates if
          other != candidate
   )
]
return front
```

Every candidate checked against every other candidate; only the non-dominated ones survive.

The Real Pareto Front



All 12 distinct (violations, afternoon count) pairs; three survive the dominance filter: $(1, 2)$, $(2, 1)$, $(4, 0)$.

3 • Participation Quick Check: Two Lexicographic Steps

Minimizing total violations gives an optimal value of 1. Write, in order, the two steps of a lexicographic solve that then minimizes the afternoon count.

Step 1 – solve `model.Minimize(total_violations)`, obtaining $v_1^* = 1$. Step 2 – add `model.Add(total_violations == 1)` on a fresh model, then solve `model.Minimize(afternoon_count)` on that constrained model.

SECTION 4

Unit 4 — Penalty Variables and Soft Global Constraints

2 • Content Soft Global Constraints: The General Recipe

Definition: Soft global constraint

A global constraint, such as `AllDifferent`, `NoOverlap` or `Cumulative`, whose satisfaction is reified into a penalty term rather than enforced as a hard rule.

Analogy

A shared lab's equipment sign-up sheet with a posted norm instead of a locked calendar: "please avoid double-booking the microscope" is requested, not enforced. Double-bookings get tallied and discouraged – a penalty – rather than made impossible outright.

The same reify-weight-sum pattern from Unit 2, now applied to a whole global constraint's satisfaction rather than one pairwise inequality.

Worked Case: Nurse Preferences — Hard Constraints

3 nurses, 2 days, 2 shifts/day. Two rules always hold:

```
for day in DAYS:
   for shift_type in SHIFT_TYPES:
      model.Add(sum(shift[nurse, day, shift_type] for nurse in NURSES
         ) == 1)
for nurse in NURSES:
   for day in DAYS:
      model.Add(sum(shift[nurse, day, shift_type] for shift_type in
         SHIFT_TYPES) <= 1)
```

Every shift-slot needs exactly one nurse; no nurse works both shifts on the same day – plain `model.Add(...)`, no reification, since neither rule may ever be violated.

Code: Reifying Each Nurse's Preference

```
pref_violated = {}
for nurse in NURSES:
   for day in DAYS:
      violated = model.NewBoolVar(f"pref_violated_{nurse}_{day}")
      model.Add(shift[nurse, day, "evening"] == 1).OnlyEnforceIf(
         violated)
      model.Add(shift[nurse, day, "evening"] == 0).OnlyEnforceIf(
         violated.Not())
      pref_violated[(nurse, day)] = violated
return pref_violated
```

All three nurses prefer to avoid evenings, but by how much differs: N1 weight 3, N2 weight 2, N3 weight 1.

Common Pitfall: One Shared Penalty Variable?

Common Pitfall

A penalty variable always ties to *one* nurse's stated preference on *one* day, never a pooled or averaged one. A single shared Boolean for all three nurses would make it impossible for the objective to route the unavoidable violation toward the cheapest nurse to disappoint.

Code: The Weighted Penalty Objective

```
total_penalty = sum(
   PREFERENCE_WEIGHTS[nurse] * violated for (nurse, _day), violated
      in pref_violated.items()
)
model.Minimize(total_penalty)
return total_penalty
```

Each nurse's weight multiplies their own violated-preference Boolean – the same reify-weight-sum pattern as Unit 2, now on a whole roster.

Code: Solving and Reporting the Roster

```
solver = cp_model.CpSolver()
solver.parameters.num_workers = 1
solver.parameters.random_seed = 42
status = solver.Solve(model)
print(f"Status: {solver.StatusName(status)}")
for nurse in NURSES:
    row = []
    for day in DAYS:
        worked = [s for s in SHIFT_TYPES if solver.Value(shift[nurse, day,
            s])]
        row.append(f"{day}:{worked[0] if worked else 'off'}")
    print(f"{nurse}: {' '.join(row)}")
print(f"Total penalty: {solver.Value(total_penalty)}")
```

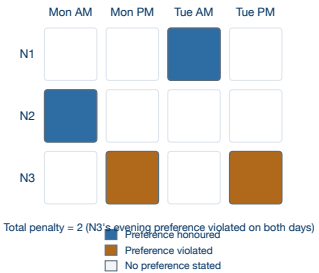
One solve call prints the schedule grid and the total penalty.

Output: The Nurse Roster, For Real

Expected Output

```
Status: OPTIMAL
N1: Mon:off  Tue:morning
N2: Mon:morning  Tue:off
N3: Mon:evening  Tue:evening
Total penalty: 2
```

The Schedule, Colour-Coded by Preference



N1 and N2 never work an evening – both preferences fully honoured; N3, the cheapest nurse to disappoint (weight 1), absorbs both evening slots for penalty $1 \times 2 = 2$ – not eliminating the unavoidable violation, but routing it to where it costs the least.

3 • Participation Quick Check: Complete the Reification

A partially written script has hard coverage in place but is missing the reified penalty variable for one nurse's evening preference. What two calls, and what objective term, complete it?

`model.Add(shift[nurse][slot] == 1).OnlyEnforceIf(pref_violated)` and `model.Add(shift[nurse][slot] == 0).OnlyEnforceIf(pref_violated.Not())`, then add `weight * pref_violated` to `model.Minimize(sum(...))` – the completed run reports total penalty 2.

SECTION 5

Summary

Summary

Summary

- Over-constrained problems relax hard constraints into Max-CSP (count violations) or WCSP (weight them) instead of declaring `INFEASIBLE` and stopping
- CP-SAT's `model.Minimize()` needs reified violation Booleans – `OnlyEnforceIf` in *both* directions – a raw condition is not a variable
- Lexicographic optimization pins each objective's optimum before the next; Pareto dominance instead hands you every trade-off point
- A complete soft-constraint model keeps coverage rules hard and preferences soft – routing violations to where they cost least

4 • Assessment Check Yourself 5 • Follow-through Transfer

One Check Yourself question closes each unit – answers collected at the chapter's end: a 4-researcher lab-equipment Max-CSP; why `model.Minimize(x != y)` fails directly; why skipping the pin breaks a lexicographic solve; whether lower penalty guarantees fairness.

- **Week 9** is this course's Midterm 1 Review – this week's models exercised as concept-map/worked-problem review over Weeks 1–8
- **Week 10** asks how much relaxing a hard constraint's right-hand side changes an LP/MIP's optimal value – not which soft constraint to violate
- **Weeks 12 and 15** return to this timetabling/rostering pair – interval-variable scheduling, then infeasibility diagnosis

In the Lab

This week's lab sheet

You run and verify the over-constrained timetable and nurse-preference models yourself, extend the five-talk model with one more reified constraint, identify the Pareto-optimal subset from a real solver run, and close by building a complete soft-constraint CP-SAT model from scratch for a new over-constrained scenario.

- Next: Midterm 1 Review, covering Weeks 1–8
- Week 10 grows Week 7's LP relaxation into duality, shadow prices and a full what-if analysis